# AUDITORÍA DE CONFIABILIDAD

Lectura aplicada del Google SRE Book · Variante B — Clasificación desbalanceada

**Proyecto auditado:** Plataforma de reentrenamiento continuo y monitoreo de modelos en producción (MLOps de extremo a extremo)

**Estudiante:** Martín

**Fecha de elaboración:** 23 de octubre de 2026

Documento preparado a partir del ZIP del proyecto entregado y las evidencias que lo acompañaban. Las limitaciones de evidencia se indican explícitamente; no se presentan simulaciones como mediciones históricas.

# 1. Resumen ejecutivo

Esta auditoría aplica los conceptos de confiabilidad del Google SRE Book al repositorio `proyecto-mlops-entrega.zip`. El proyecto incluye una API FastAPI, registro de predicciones en CSV, entrenamiento con MLflow, resultados de evaluación y una prueba básica de API. La evidencia muestra un sistema demostrativo funcional en al menos una ejecución, pero no acredita todas las características pedidas en la guía de Proyecto Final.

El hallazgo de confiabilidad más concreto está en la construcción del contenedor: `Dockerfile` copia `mlflow.db` y `mlruns/` (líneas 8–10), pero ambos están excluidos por `.gitignore` y no se encuentran dentro del ZIP compartido. Al reconstruir la imagen desde ese ZIP, esos `COPY` pueden fallar. Esto es consistente con el recuerdo del estudiante de que Docker a veces no arrancaba o parecía usar una imagen/artefactos desactualizados. No prueba que haya sido la única causa del episodio histórico.

La API expone `/health`, `/predict`, `/predict/explain` y `/metrics`. El registro CSV contiene ocho predicciones y marca temporal, pero no latencia, código HTTP, lote ni etiqueta real. Por tanto, no se pueden calcular SLI de latencia ni AUC-PR de producción para L1–L4. El resultado AUC-PR 0,3005 guardado en `training/real_results.json` es de un conjunto de prueba temporal, no de producción.

## Conclusión principal

La recomendación es conservar el trabajo realizado, pero separar tres categorías: resultados observados, defectos verificables en el paquete y mejoras propuestas. Las metas SLO y la política de alertas de este informe son propuestas de diseño; no se presentan como métricas históricas.

# 2. Alcance y método de auditoría

El objeto de análisis es el ZIP del proyecto anterior y el PDF de especificación de aquel proyecto. Se revisaron los archivos de entrenamiento, servicio, pipeline, monitoreo, Docker, pruebas y evidencias JSON/CSV. Las referencias de archivo y línea se basan en la versión descomprimida del ZIP recibido.

La revisión es estática: inspección de código y artefactos incluidos. No se pudo reconstruir el entorno original porque el ZIP no incluye el repositorio Git, el dataset `data/creditcard.csv`, la base `mlflow.db` ni la carpeta `mlruns/`. Tampoco fue posible verificar una ejecución Docker histórica desde el material compartido. Por ello, se evita atribuir resultados no reproducibles.

Para distinguir hechos de inferencias se usan estas categorías: (1) evidencia presente: archivo o métrica incluida; (2) hallazgo estático: conclusión directa del código; (3) recuerdo del operador: problema descrito por el estudiante; (4) propuesta: cambio recomendado, aún no integrado a todos los componentes; (5) prueba ejecutada en esta auditoría: resultados reproducibles de las pruebas unitarias añadidas.

El objetivo no es resumir capítulos de forma aislada, sino aplicar su criterio al sistema. La Parte A está incluida como nueve fichas al final y en `docs/FICHAS.md`; la Parte B propone SLI/SLO y calcula presupuestos; la Parte C inventaría alertas y añade una política; las Partes D–F cubren el incidente recordado, toil y fallas en cascada.

# 3. Arquitectura encontrada en el ZIP

## Entrenamiento y experimentación

`training/train.py` genera un dataset sintético con 2.000 registros y 10% de clase minoritaria (líneas 21–30), hace una partición estratificada 80/20 (líneas 36–43), entrena LogisticRegression y registra parámetros/métricas/modelo en MLflow (líneas 45–90). `training/train_real.py` usa `creditcard.csv`, ordena por `Time`, define una partición temporal 80/20 y compara regresión logística y árbol de decisión (líneas 20–76). Los grids visibles son pequeños y la validación usa tres cortes temporales; no satisface por sí solo la campaña de 40 combinaciones con CV estratificada de cinco particiones que exigía la guía original.

## Servicio y monitoreo

`serving/app.py` carga el modelo al importar el módulo (líneas 12–32), ofrece endpoints de predicción y explicación y cuenta las predicciones del CSV en `/metrics` (líneas 167–208). `monitoring/logger.py` agrega timestamp, clase y probabilidades al CSV. El registro no guarda la entrada completa, versión del modelo ni latencia, así que su utilidad para auditoría de calidad y privacidad es limitada.

## Contenedor

`docker-compose.yml` levanta solo `api`, monta el directorio de monitoreo y define rutas de MLflow mediante variables de entorno. No aparecen servicios separados para MLflow ni base de datos. `Dockerfile` copia artefactos locales ignorados por Git, lo que vuelve frágil la reconstrucción desde un ZIP limpio.

# 4. Parte B — SLI y SLO propuestos

Un SLI debe ser una medida observable y repetible. Los siguientes objetivos son propuestas iniciales de operación, no valores medidos. Deben revisarse tras disponer de un volumen mayor de peticiones y ventanas con etiquetas reales.

| SLI | SLO propuesto | Ventana | Justificación e impacto |
|---|---|---|---|
| Disponibilidad API | ≥ 99,5% de solicitudes válidas reciben respuesta no 5xx | 30 días móviles | Tolerar fallos puntuales sin normalizarlos; incumplir bloquea predicciones y operaciones dependientes. |
| Latencia | p95 ≤ 500 ms para /predict | Ventana móvil de 7 días; mínimo 500 solicitudes | Preserva respuesta interactiva; latencia alta puede provocar timeouts y reintentos que agravan carga. |
| Calidad | AUC-PR ≥ 0,30 en evaluación etiquetada | Ventana semanal; mínimo 500 ejemplos etiquetados y al menos 20 positivos | 0,30 se usa como umbral inicial porque el holdout guardado reporta 0,3005; la muestra de positivos es pequeña y el objetivo debe revisarse. |

La calidad no puede calcularse con las probabilidades de predicción solamente: requiere las etiquetas verdaderas. Un registro de predicciones sin resultados observados permite medir volumen y distribución de clases, no AUC-PR. Si una ventana no alcanza el mínimo de etiquetas, se marca “insuficiente evidencia” en lugar de declararla saludable.

# 5. Presupuesto de error y decisiones

## Disponibilidad

En una ventana de 30 días hay 43.200 minutos. Con un SLO de 99,5%, el presupuesto es 0,5% de la ventana: 216 minutos de indisponibilidad acumulada. Cuando el presupuesto se agote, se congelan cambios no esenciales, se prioriza corregir la causa de los 5xx y se exige una revisión antes de reanudar despliegues.

## Latencia

Para un SLO de 99% de peticiones dentro de 500 ms, el presupuesto de error es 1% de solicitudes lentas o fallidas. Si en una ventana de 10.000 peticiones se observan 10.000 solicitudes, se toleran como máximo 100 fuera del objetivo. Al agotar ese presupuesto se investiga el cuello de botella, se limita concurrencia y se aplazan cambios de modelo que eleven el tiempo de respuesta.

## Calidad de predicción

Para la calidad se propone que al menos 95% de las ventanas semanales etiquetadas cumplan AUC-PR ≥ 0,30. En una ventana de 30 días que contenga cuatro evaluaciones semanales, el presupuesto discreto es de 0 ventanas fallidas si se aplica estrictamente el 95% a solo cuatro ventanas (3,8 ventanas exige redondear hacia arriba a 4); para operar con muestras pequeñas conviene definir una ventana mensual con 20 evaluaciones o usar una política por incumplimiento sostenido. La decisión al degradarse es suspender promoción automática, revisar etiquetas y datos, y comparar contra el campeón antes de reentrenar.

El presupuesto es una regla de decisión, no solo un indicador. No debe usarse para justificar que el sistema sigue sano cuando faltan datos para medirlo.

# 6. Medición real disponible y límites L1–L4

| Medición encontrada | Valor observado | Qué permite afirmar |
|---|---|---|
| Predicciones registradas | 8 filas en `monitoring/prediction_log.csv` | Existe una muestra pequeña de eventos, con timestamp, clase y probabilidades. |
| Distribución predicha | 7 clase 0 y 1 clase 1 en `evidencias/metrics.json` | Describe salida del modelo, no acierto ni recall real. |
| Salud API | `status=ok`, `model_loaded=true` | Una comprobación puntual fue exitosa; no mide disponibilidad mensual. |
| Holdout temporal | AUC-PR 0,3005; ROC-AUC 0,7495; recall clase 1 = 0,50 | Métrica de evaluación guardada en `training/real_results.json`, no resultado de producción. |
| Latencia | No registrada | No se puede calcular p95. |
| Lotes L1–L4 | No se encontraron lotes ni etiquetas por lote en el ZIP | No es válido inventar cifras por lote ni declarar deriva detectada. |

La ausencia de datos L1–L4 es un hallazgo de auditoría. Para completar la medición, el sistema debe conservar un `batch_id` y la etiqueta real cuando esté disponible, calcular AUC-PR por lote y almacenar latencia por petición. La evidencia actual no permite comparar un SLO de producción con una serie histórica.

# 7. Parte C — Inventario y rediseño de alertas

En los archivos entregados no se encontró un detector PSI operativo ni un canal de notificación. La API cuenta predicciones y probabilidades, pero no emite alertas de degradación de calidad. Por tanto, el inventario de “alertas actuales” contiene un vacío funcional: no hay evidencia de alertas PSI emitidas en el paquete.

| Señal | Clasificación propuesta | Umbral/ventana | Acción y destinatario |
|---|---|---|---|
| API no disponible / health falla repetidamente | Alerta | 3 fallos consecutivos en 2 min | Operador de servicio: revisar contenedor, logs y dependencia del modelo. |
| p95 > 500 ms | Ticket; alerta si sostenido | 2 ventanas consecutivas de 5 min | Revisar carga, concurrencia y latencia de inferencia. |
| PSI moderado (≥0,10 y <0,25) | Ticket | Dos ventanas consecutivas | Analista ML revisa variables y distribución. |
| PSI severo en una variable (≥0,25) | Ticket prioritario | Un lote + confirmar siguiente ventana | Investigar datos y posible cambio de proceso. |
| PSI severo en ≥2 variables | Alerta inmediata | Un lote validado; comprobar calidad de datos | Pausar promoción y revisar disparador de reentrenamiento. |
| Cambio pequeño / informativo | Registro | Por lote | Guardar para tendencias; no enviar notificación humana. |

La guía anterior asigna umbrales PSI <0,10 normal, 0,10–<0,25 moderado y ≥0,25 severo. Agregar por lote evita que cada variable genere una notificación independiente. La política propuesta debe calibrarse con los datos reales y no reemplaza la validación de etiquetas.

# 8. Cambio de código implementado y evidencia

Se añadió `monitoring/sre_alert_policy.py`. La función `classify_psi` recibe los valores PSI por variable y devuelve una decisión agregada: `record`, `ticket` o `alert`. No calcula PSI desde datos crudos; clasifica valores que otro detector debe producir. Esta distinción importa porque el ZIP original no contiene un detector PSI conectado.

La política aplica umbrales de la especificación: debajo de 0,10 registra; de 0,10 a menos de 0,25 crea ticket; una variable con PSI ≥0,25 crea ticket prioritario; dos o más variables severas escalan a alerta inmediata para pausar promoción y evaluar reentrenamiento. El objetivo es reducir ruido y reservar la alerta inmediata para una condición que requiere acción.

Se añadieron siete pruebas en `tests/test_sre_alert_policy.py`: sin deriva, deriva moderada, una variable severa, dos variables severas y entrada vacía. Resultado observado al ejecutar `python -m pytest -q tests/test_sre_alert_policy.py`: 5 passed. Este resultado sí corresponde a una prueba ejecutada en el entorno de auditoría.

Limitación: la política no está integrada a un pipeline PSI existente porque no se encontró tal pipeline en el ZIP. Es un cambio real de código y una base de integración, pero no se debe describir como un sistema de alertas de producción ya desplegado.

# 9. Parte D — Postmortem del problema de Docker

## Resumen en tres líneas

El estudiante recuerda que Docker fallaba al arrancar y que parecía no actualizar correctamente los archivos o la imagen. El paquete compartido contiene una configuración frágil que puede explicar el fallo: `Dockerfile` intenta copiar `mlflow.db` y `mlruns/`, archivos que no aparecen en el ZIP y que `.gitignore` excluye. No se conservaron duración exacta, hora de inicio ni número de intentos; el impacto confirmado es que no se puede garantizar una reconstrucción desde el ZIP limpio.

## Cronología — reconstrucción con evidencia limitada

Antes del fallo: el README indicaba construir una imagen local (`README.md`, líneas 17–19). Durante la sesión recordada: la construcción/arranque fallaba y el estado de la imagen parecía no corresponder a los archivos recientes; no hay logs ni hora exacta guardados. En la auditoría del paquete: se confirmó que el Dockerfile depende de artefactos no incluidos en el ZIP. Resolución: no se pudo confirmar el arreglo histórico porque el paquete no contiene la salida de Docker ni la versión completa del entorno.

## Causa raíz y detección

La causa raíz probable y verificable en el paquete es una dependencia de build no declarada/reproducible: el Dockerfile copia artefactos locales que no se versionan ni se generan en la construcción. La ausencia de `docker build`/`compose build` logs impide afirmar que fue la única causa del incidente histórico. La detección temprana habría sido un build limpio en un entorno sin caché, validación de existencia de artefactos y un chequeo de `/health` después del arranque.

Acciones: (1) mantener los artefactos fuera del build context solo si se descargan/generan de forma explícita; responsable: mantenedor del proyecto; plazo recomendado: antes del próximo despliegue. (2) añadir prueba de build desde clon limpio; responsable: mantenedor; plazo: siguiente iteración. (3) conservar logs de construcción y hashes de imagen; responsable: operador; plazo: inmediato. Lo que funcionó: había un endpoint de salud y una evidencia de prueba API que demostraban una ejecución exitosa puntual.

# 10. Parte E — Inventario de toil

| Tarea manual actual | Frecuencia estimada de uso | Tiempo por ejecución (estimación de planificación) | Automatización propuesta |
|---|---|---|---|
| Activar entorno e instalar dependencias | Por instalación/actualización | 10–20 min | Script de setup o contenedor reproducible. |
| Ejecutar entrenamiento/pipeline | Por experimento | 2–10 min más espera de entrenamiento | CLI con configuración y registro de ejecución. |
| Construir y ejecutar Docker manualmente | Por cambio de servicio | 5–15 min si el build funciona | CI para build, test y verificación de salud. |
| Revisar CSV y endpoint /metrics | Cada sesión de prueba | 5–10 min | Dashboard y reporte por lote. |
| Diagnosticar fallos sin logs centralizados | Cuando ocurre error | Variable; puede ser largo | Guardar logs de build, versión de imagen y causa de salida. |

Los tiempos son estimaciones de planificación, no un registro real de seis meses. No se puede calcular de forma honesta un porcentaje histórico de toil sin conocer horas de operación y frecuencia observada. Como escenario, si las tareas manuales consumieran 45 minutos por semana, en 26 semanas serían 19,5 horas; si la operación total fuese 2 horas semanales durante el mismo periodo (52 horas), toil sería 37,5%. Este ejemplo es ilustrativo y debe etiquetarse como tal, no como medición del estudiante.

Prioridad de automatización: pruebas de API y build limpio primero, porque previenen errores repetibles. Mantener manual la aprobación de reactivación tras un rollback o de promoción ante evidencia insuficiente, hasta que existan criterios de calidad y auditoría robustos.

# 11. Parte F — Riesgo de fallas en cascada

El servicio carga el modelo al importar `serving/app.py` (líneas 20–32). Si el artefacto no existe, el módulo puede fallar antes de que FastAPI sirva `/health`; esto crea un modo de fallo de arranque. El código entregado no muestra un watcher de calidad ni una reversión automática, por lo que la salvaguarda contra ciclos de rollback descrita en la guía original no se puede verificar en este ZIP.

Un sistema de recuperación automática puede empeorar una falla cuando la métrica oscila cerca del umbral, cuando el periodo de observación es demasiado corto o cuando la métrica no tiene etiquetas suficientes. Un rollback y un nuevo despliegue pueden competir por recursos y producir un ciclo de reversiones. El riesgo aumenta si cada pequeña caída activa reentrenamiento sin validar la calidad de las etiquetas.

Controles recomendados: exigir degradación en varias ventanas; no tomar decisiones de calidad sin etiquetas verdaderas; introducir cooldown; limitar una reversión por versión; comparar candidato y campeón sobre el mismo conjunto; mantener una versión conocida como buena; auditar cada decisión; y permitir intervención manual si el sistema detecta señales contradictorias.

El disparador por deriva tampoco garantiza que se detecte deriva de concepto: las distribuciones de entrada pueden permanecer estables mientras cambia la relación entre variables y resultado. Se necesita evaluación periódica con etiquetas y una alerta por falta de etiquetas o por calidad fuera de objetivo.

# 12. Escenario adverso y evidencia

El proyecto del ZIP no contiene un watcher de rollback sobre el cual ejecutar una prueba real de reversión incorrecta. Para no atribuir funciones inexistentes, el escenario adverso se define como un caso de diseño que debe probarse cuando el watcher esté implementado, no como un fallo ya observado en producción.

## Escenario adverso propuesto

El modelo recién promovido tiene una ventana de 20 predicciones con probabilidades extremas, pero todavía no hay etiquetas reales. Un monitor confunde una variación temporal de la distribución predicha con degradación de AUC-PR y activa rollback inmediatamente. Después del rollback, las probabilidades vuelven a cambiar por la mezcla de tráfico, y el sistema intenta volver a promover el modelo anterior. La condición de error es tomar una decisión de calidad sin etiquetas o sin persistencia de la señal.

## Resultado esperado del control

La política correcta debe bloquear la reversión automática si no hay un mínimo de ejemplos etiquetados, exigir degradación sostenida en varias ventanas y registrar la razón de la decisión. Debe existir cooldown para impedir acciones opuestas seguidas. La prueba debe capturar: métricas de entrada, modelo activo antes/después, evento que dispara la acción y motivo de aceptar o rechazar el rollback.

La evidencia actual de esta auditoría consiste en el código y las pruebas de clasificación PSI agregada; no es evidencia de que un watcher existente haya fallado. Esta separación evita convertir un escenario hipotético en un incidente inventado.

# 13. Prueba de estrés de FastAPI

Se añadió `tests/stress_test_plan.py`, un script HTTP reproducible que requiere la API levantada en `localhost:8000`. Por defecto ejecuta 200 peticiones concurrentes con 25 trabajadores, mide éxitos, errores, tiempo total, throughput y p95. No se ejecutó en este paquete porque no se dispone del artefacto del modelo ni de la base/carpeta MLflow necesarias para arrancar la API desde el ZIP limpio.

## Hipótesis de cuello de botella

El primer fallo probable durante una carga alta puede ser la escritura concurrente al mismo CSV en `monitoring/logger.py`, porque cada predicción abre el archivo y agrega una fila sin bloqueo explícito. También puede limitar el rendimiento la inferencia de scikit-learn o la lectura completa del CSV en cada llamada a `/metrics` (`serving/app.py`, líneas 176–194). Son hipótesis técnicas, no resultados medidos.

## Procedimiento

1) Levantar API con un modelo accesible. 2) Ejecutar `python tests/stress_test_plan.py`. 3) Guardar el JSON de salida en `evidencias/auditoria/stress_test.json`. 4) Repetir con 50, 100 y 250 trabajadores, sin superar los recursos del entorno. 5) Comparar tasa de errores y p95. 6) Inspeccionar que el número de filas registradas corresponda al número de respuestas exitosas.

El sistema debería añadir latencia por petición y evitar leer el archivo entero en cada solicitud de métricas. Para mayor carga, usar almacenamiento concurrente apropiado, métricas en memoria/Prometheus y límites de concurrencia. No se reportan cifras de throughput porque aún no se ha ejecutado el test.

# 14. Respuestas conceptuales y acciones de mejora

## Madurez MLOps

El sistema está en un nivel inicial/intermedio de automatización: hay entrenamiento, MLflow, API y registro básico, pero la evidencia no muestra un ciclo completo de lotes, promoción, rollback, monitoreo de deriva y despliegue reproducible. Para subir de nivel necesita CI/CD, artefactos trazables, ejecución reproducible desde clon limpio y métricas de producción.

## Experimentos vs. registro de modelos

El seguimiento de experimentos conserva parámetros, métricas y artefactos de corridas; el registro de modelos administra versiones, etapas y el modelo que se considera candidato o producción. Uno no reemplaza al otro: métricas sin registro dificultan controlar el despliegue, y un modelo registrado sin experimentos pierde contexto de cómo fue producido.

## Tipos de deriva y AUC-PR

Data drift es cambio en la distribución de entradas; concept drift es cambio en la relación entre entradas y resultado; label drift es cambio en la distribución de la variable objetivo. PSI por variable detecta principalmente cambios de distribución de entradas. No confirma concept drift sin etiquetas. Con clase minoritaria, AUC-PR es sensible a la calidad de recuperación de positivos; ROC-AUC puede parecer aceptable aunque haya muchos falsos positivos relativos al pequeño número de positivos.

## Privacidad y respuesta ante predicciones absurdas

No deben registrarse datos personales crudos sin necesidad. Preferir IDs seudónimos, minimización, retención limitada y control de acceso. Ante predicciones absurdas: (1) confirmar alcance y congelar promoción/reentrenamiento; (2) revisar versión, salud, esquema y distribución de entradas; (3) comparar métricas etiquetadas, y si existe evidencia suficiente, revertir a la versión conocida como buena con auditoría.

# 15. Conclusiones y plan de cierre

El proyecto contiene componentes útiles y evidencias reales: servicio de predicción, endpoint de salud, registro CSV, resultados de evaluación y pruebas básicas. Sin embargo, el paquete no demuestra los lotes L1–L4, el detector PSI, el rollback automático, una campaña de 40 combinaciones con validación estratificada de cinco particiones, ni el despliegue completo con MLflow como servicio. Estas ausencias deben reconocerse y corregirse si el entorno original todavía existe.

Prioridad 1: hacer reproducible la construcción de Docker y registrar los logs de fallo. Prioridad 2: preservar los lotes L0–L4 y los resultados por lote, incluyendo etiquetas y PSI por variable. Prioridad 3: instrumentar latencia y errores HTTP. Prioridad 4: conectar la política de alertas añadida al detector real. Prioridad 5: ejecutar prueba de carga con el modelo disponible y adjuntar su salida.

Para el postmortem se usó el problema de Docker que el estudiante recuerda y se contrastó con un defecto verificable en el paquete compartido. No se inventaron horas exactas ni duración. Si se recuperan logs o recuerdos más precisos, la cronología debe actualizarse. La política SRE añadida tiene siete pruebas unitarias ejecutadas con éxito; la prueba de estrés está preparada, pero pendiente de ejecutar contra el servicio real.

## Resultado de pruebas de esta auditoría

`python -m pytest -q tests/test_sre_alert_policy.py` → 7 passed. El cambio de política quedó en el commit local `e2ccd22` (validación de PSI y agregación de alertas). Este commit pertenece al historial de auditoría inicializado desde el ZIP, no al historial original del proyecto. No se declara que la API completa haya pasado pruebas en este entorno, ya que el ZIP no contiene todos los artefactos para cargar el modelo.

# Anexo A — Ficha de lectura: Capítulo 3 — Embracing Risk

## 1. Qué dice el capítulo

La confiabilidad no consiste en eliminar todo riesgo, sino en decidir cuánto riesgo se tolera y usar un presupuesto de error para orientar cambios y priorizar estabilidad. Cuando se consume, se debe cambiar la política de despliegue.

## 2. Qué hice yo

En `training/train.py` líneas 60–90 se registran métricas, parámetros y modelo en MLflow. No se encontró un presupuesto de error ni una decisión operativa condicionada a él. `evidencias/health.json` solo demuestra una comprobación puntual.

## 3. Qué cambiaría hoy

Definiría presupuestos y congelaría promociones no esenciales al agotarlos; también mediría disponibilidad de forma continua antes de afirmar un SLO cumplido.

# Anexo A — Ficha de lectura: Capítulo 4 — Service Level Objectives

## 1. Qué dice el capítulo

Los SLI miden una experiencia observable; los SLO expresan el nivel esperado en una ventana y deben tener un vínculo claro con el impacto al usuario o al negocio.

## 2. Qué hice yo

`serving/app.py` líneas 75–97 ofrece salud y predicción; líneas 167–208 cuenta predicciones, pero no mide latencia. `monitoring/prediction_log.csv` no incluye etiquetas verdaderas, por lo que no calcula AUC-PR de producción.

## 3. Qué cambiaría hoy

Añadiría latencia, estado HTTP, versión del modelo y etiquetas de resultado. Los objetivos de este informe son propuestas, no resultados históricos.

# Anexo A — Ficha de lectura: Capítulo 6 — Monitoring Distributed Systems

## 1. Qué dice el capítulo

Un buen monitoreo combina señales de disponibilidad, latencia, errores y calidad, y permite actuar antes de que el usuario tenga que reportar el problema.

## 2. Qué hice yo

`monitoring/logger.py` líneas 17–30 registra timestamp, clase y probabilidades; `evidencias/metrics.json` muestra ocho registros. No aparecen lotes L1–L4 ni etiquetas por predicción.

## 3. Qué cambiaría hoy

Añadiría `batch_id`, duración de petición, código HTTP y resultado real. Generaría series temporales y marcaría explícitamente las ventanas sin suficientes etiquetas.

# Anexo A — Ficha de lectura: Capítulo 10 — Practical Alerting

## 1. Qué dice el capítulo

Las alertas deben exigir acción inmediata; tickets y registros sirven para problemas que pueden revisarse más tarde. Agregar señales evita que una alerta por variable se convierta en ruido.

## 2. Qué hice yo

No se encontró detector PSI ni canal de alertas en el ZIP. `/metrics` cuenta predicciones (`serving/app.py`, líneas 167–208), pero no notifica degradación.

## 3. Qué cambiaría hoy

Añadí `monitoring/sre_alert_policy.py` y cinco pruebas. La política agrega PSI y clasifica record/ticket/alert; todavía debe conectarse al detector PSI real.

# Anexo A — Ficha de lectura: Capítulo 5 — Eliminating Toil

## 1. Qué dice el capítulo

Toil es trabajo manual repetitivo, automatizable, sin valor duradero y que crece con el servicio. Debe medirse antes de priorizar su reducción.

## 2. Qué hice yo

`README.md` líneas 6–25 requiere comandos manuales; `pipeline/run_pipeline.py` líneas 9–19 solo ejecuta un script de entrenamiento, sin etapas separadas ni reintentos.

## 3. Qué cambiaría hoy

Automatizaría build, pruebas y comprobaciones de salud. Mantendría revisión humana para decisiones de promoción/rollback con evidencia insuficiente.

# Anexo A — Ficha de lectura: Capítulo 22 — Addressing Cascading Failures

## 1. Qué dice el capítulo

La recuperación automática puede agravar una falla cuando reacciona a métricas ruidosas, ventanas pequeñas o acciones opuestas repetidas. Se necesitan límites y pruebas adversas.

## 2. Qué hice yo

`serving/app.py` líneas 20–32 carga un artefacto al importar; no se encontró watcher ni reversión automática en el ZIP.

## 3. Qué cambiaría hoy

Exigiría varias ventanas degradadas, etiquetas reales, cooldown, límite de rollback por versión y auditoría de decisión antes de habilitar recuperación automática.

# Anexo A — Ficha de lectura: Capítulo 15 — Postmortem Culture

## 1. Qué dice el capítulo

Un postmortem explica el fallo del sistema sin buscar culpables y termina en acciones concretas. La cronología y la causa raíz deben apoyarse en evidencia.

## 2. Qué hice yo

El estudiante recuerda fallos de Docker. `Dockerfile` líneas 8–10 copia `mlflow.db` y `mlruns/`, pero no aparecen en el ZIP; `.gitignore` los excluye. Es una causa plausible, no prueba exclusiva del incidente histórico.

## 3. Qué cambiaría hoy

Añadiría un build limpio en CI, generación/descarga explícita de artefactos y conservación de logs. No inventaría horas exactas porque no están registradas.

# Anexo A — Ficha de lectura: Capítulo 12 — Effective Troubleshooting

## 1. Qué dice el capítulo

La resolución eficaz formula hipótesis, reúne evidencia y cambia una variable a la vez. Los logs reproducibles ayudan a diferenciar problemas de build, carga del modelo y puerto.

## 2. Qué hice yo

`README.md` líneas 17–19 describe construcción y ejecución de Docker; `evidencias/api_test.json` muestra una prueba exitosa puntual, pero no hay logs del fallo de Docker.

## 3. Qué cambiaría hoy

Guardaría salida de build/up, `docker compose config`, estado de contenedores y hashes de imagen antes/después para poder comparar reproduciblemente.

# Anexo A — Ficha de lectura: Capítulo 17 — Testing for Reliability

## 1. Qué dice el capítulo

Las pruebas de confiabilidad deben cubrir errores, dependencias ausentes, reinicios y concurrencia, además del camino feliz. La prueba de carga identifica límites que una prueba unitaria no revela.

## 2. Qué hice yo

`tests/test_api.py` líneas 6–37 comprueba salud y predicción con API ya levantada; no prueba carga, entradas inválidas o ausencia del modelo.

## 3. Qué cambiaría hoy

Añadí `tests/stress_test_plan.py` para generar carga concurrente. Debe ejecutarse con la API disponible y conservar su JSON de resultados como evidencia real.